

Kennesaw State University
Parallel and Distributed Computing

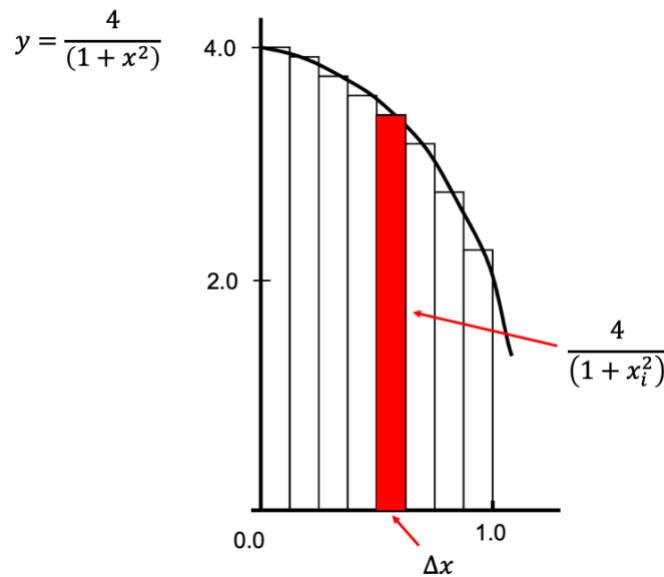
Project – MPI

Instructor: Kun Suo
Points Possible: 100
Difficulty: ★★★★★☆

Part A (50 points): Point-to-point communication

Mathematically, we know the following equation:

$$\int_0^1 \frac{4}{(1+x^2)} dx = \pi$$



We can approximate the value of π as a sum of rectangles:

$$\sum_{i=0}^N f(x_i) \Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i .

The following code implements the above calculation of π . We divide the area between 0 and 1 into 1,000,000 small rectangles and the value of π is approximately equal to the sum of all rectangles' size. However, the program executes in the sequential implementation.

<https://github.com/kevinsuo/CS4504/blob/master/pi.c>

```
-----  
#include <stdio.h>  
#include <stdlib.h>  
#include <time.h>  
#define NUMSTEPS 1000000  
  
int main() {  
    int i;  
    double x, pi, sum = 0.0;  
    struct timespec start, end;  
  
    clock_gettime(CLOCK_MONOTONIC, &start);  
    double step = 1.0/(double) NUMSTEPS;  
    x = 0.5 * step;  
  
    for (i=0; i<= NUMSTEPS; i++){  
        x+=step;  
        sum += 4.0/(1.0+x*x);  
    }  
    pi = step * sum;  
    clock_gettime(CLOCK_MONOTONIC, &end);  
    u_int64_t diff = 1000000000L * (end.tv_sec - start.tv_sec) + end.tv_nsec -  
start.tv_nsec;  
  
    printf("PI is %.20f\n", pi);  
    printf("elapsed time = %llu nanoseconds\n", (long long unsigned int) diff);  
  
    return 0;  
}  
-----
```

- (1) Write a parallel program to calculate π using MPI based on this sequential solution. Implement the parallel π calculation using **MPI_Send** and **MPI_Recv** for Point-to-point communication. Do not use MPI collective operations in Part A.

To compile the program with MPI, use: (here file refers to your code name)

`$ mpicc -g file.c -o file.o`

Please write a brief report introducing your implementation.

- (2) Estimate your MPI program execution time and compare the time using different number of threads (n=1, 2, 4, 8, ...).

One possible expected output is (here my executable file is named as pi-mpi.o):

```
administrator@ubuntu1804vm ~> mpiexec -n 2 pi-mpi.o
PI is 3.14159165358979830529
elapsed time = 7463985 nanoseconds
administrator@ubuntu1804vm ~> mpiexec -n 4 pi-mpi.o
PI is 3.14158965357486108516
elapsed time = 7745198 nanoseconds
administrator@ubuntu1804vm ~> mpiexec -n 8 pi-mpi.o
PI is 3.14158565355710805989
elapsed time = 9736663 nanoseconds
```

Part B (50 points): Collective communication

Refactor the same calculation in Part A using MPI collective operations. Your program must use **MPI_Bcast** or **MPI_Reduce**, or one additional collective with a meaningful purpose.

Use the same value of N for Parts A and B. Choose N large enough that the one-rank run lasts long enough to measure reliably. Run every configuration three times, calculate the average of the three runs, and compare the performance in part A and B.

Implementation	Ranks	Run 1	Run 2	Run 3	Average
Part A	1				
Part A	2				
Part A	4				
Part A	8				
Part B	1				
Part B	2				
Part B	4				
Part B	8				

Submitting Assignment

Submit your assignment file through D2L using the appropriate link.

The submission must include the *source code*, and *a report describe your code logic. Output screenshot of your code* should be included in the report.